

PAD (PPGCC) - Programação em MPI

A atividade pode ser realizado de duas formas/opções possíveis:

Opção 1: Trazer código-fonte de interesse

Isolar e preparar um trecho (considerado pequeno) de um código-fonte de interesse (para projeto de pesquisa da pós-graduação) em C/C++ ou Fortran, com seu respectivo conjunto de dados de entrada, cuja execução gaste, pelo menos, na casa de dezenas de segundos. Otimizar o código por MPI e fazer medidas de desempenho relacionadas ao tempo de execução e *speedup*.

A entrega da opção 1 consiste em um **arquivo compactado** contendo:

- A. Dois ou mais **arquivos relacionados ao código-fonte utilizado, o conjunto de dados** de entrada e,
- B. Um relatório que deve conter uma breve descrição sobre o código-fonte utilizado, explicando para qual finalidade ele foi criado, e os dados das medidas de desempenho citadas.

Opção 2: Jogo da Vida em MPI

O Jogo da Vida¹, criado por John H. Conway, utiliza um autômato celular para simular gerações sucessivas de uma sociedade de organismos vivos. É composto por um tabuleiro bidimensional, infinito em qualquer direção, de células quadradas idênticas. Cada célula tem exatamente oito células vizinhas (todas as células que compartilham, com a célula original, uma aresta ou um vértice). Cada célula está em dois possíveis estados: viva ou morta (correspondentes aos valores 1 ou 0). Uma geração da sociedade é representada pelo conjunto de estados das células do tabuleiro. Sociedades evoluem de uma geração para a próxima aplicando simultaneamente, a todas as células do tabuleiro, regras que estabelecem o próximo estado de cada célula. As regras são:

- a. Células vivas com menos de 2 (dois) vizinhas vivas morrem por abandono;
- b. Cada célula viva com 2 (dois) ou 3 (três) vizinhos deve permanecer viva para a próxima geração;
- c. Cada célula viva com 4 (quatro) ou mais vizinhos morre por superpopulação.
- d. Cada célula morta com exatamente 3 (três) vizinhos deve se tornar viva.

Considere um tabuleiro finito, $N \times N$ com bordas infinitas, ou seja, a fronteira esquerda liga-se com a fronteira direita e a fronteira superior liga-se com a fronteira inferior. Admita que $(0,0)$ identifica a célula no canto superior esquerdo do tabuleiro e que $(N-1,N-1)$ identifica a célula no canto inferior direito.

Estruture seu programa da forma abaixo:

- A. Aloque dinamicamente a(s) matriz(es) necessária(s) (de números inteiros) para representar duas gerações do tabuleiro de tamanho $N \times N$. Sugestão: use nomes para as matrizes (*arrays*) como *grid* e *newgrid* para a geração atual e futura, respectivamente.
- B. Inicie cada posição da geração inicial do tabuleiro (*array*) pseudo-aleatoriamente da seguinte forma:

```
#define SRAND_VALUE 1985

srand(SRAND_VALUE);

for(i = 0; i<dim; i++) { //laço sobre as células do tabuleiro sem contar com
um eventual halo

    for(j = 0; j<dim; j++) {

        grid[i][j] = rand() % 2;

    }

}
```

- C. Crie uma função/método que retorne a quantidade de vizinhos vivos de cada célula na posição i,j :

```
int getNeighbors(int** grid, int i, int j) { ... }
```

- D. Crie um laço de repetição para executar um número máximo de iterações do jogo da vida, ou seja, esse laço deverá permitir que se compute a quantidade de gerações sucessivas do tabuleiro que devem ser geradas.
- E. Crie um procedimento (ou trecho de código) que, ao finalizar todas as iterações/gerações, some todas as posições da última geração do tabuleiro e retorne a quantidade de células vivas.

Deve-se considerar um tabuleiro quadrado de dimensões 2048*2048 e um total de 2000 gerações sucessivas do tabuleiro.

A quantidade de células vivas esperadas para as gerações iniciais e última geração, sob as condições citadas, são as seguintes:

Condição inicial: 2096241

Geração 1: 1146561

Geração 2: 1063629

Geração 3: 1052114

Geração 4: 1000392

...

Última geração (2000 iterações): 146951 células vivas

A medida de desempenho deve contemplar o tempo total de execução (medido com *time*) e o tempo de execução apenas do trecho que envolve o laço que computa as gerações sucessivas. No relatório devem constar medidas de desempenho (tempo de processamento) para a versão serial e a versão paralela em MPI, variando a quantidade de processos da seguinte maneira: 1, 2, 4 e 8 processos, pelo menos (pode-se aumentar a quantidade se houver disponibilidade de processadores). Entregue o código-fonte e um relatório com o desempenho obtido.

Durante o desenvolvimento do programa pode-se utilizar tabuleiros e quantidade de gerações menores. Mas a análise de desempenho do relatório deve contar com os valores definidos acima, ou seja:

Tamanho do tabuleiro = 2048×2048

Quantidade de gerações = 2000

Quantidade mínima de processos a se avaliar: 1, 2, 4 e 8.

Os resultados podem ser demonstrados em tabelas ou gráficos.

Demonstre também que a versão paralela chega exatamente ao mesmo valor de células vivas ao final da execução.

[1] – Martin Gardner, “Mathematical Games – The fantastic combinations of John Conway’s new solitaire game life”, Scientific American 223, Oct. 1970, pp 120-123.

http://ddi.cs.uni-potsdam.de/HyFISCH/Produzieren/lis_projekt/proj_gamelife/ConwayScientificAmerican.htm

<https://playgameoflife.com/>

Dicas:

- a) Faça uma divisão de domínio para o tabuleiro do jogo da vida de forma **unidimensional**, de forma que cada processo seja responsável pelo cálculo de um pedaço da tabela que corresponde a um conjunto de linhas.
- b) os processos vizinhos deverão trocar dados entre si, correspondentes às linhas de fronteira entre as subtabelas adjacentes. Deve-se tomar cuidado para não ocorrerem *deadlocks* nesta comunicação.
- c) A inicialização da tabela/matriz **não precisa ser feita em paralelo**, mas, caso seja, deve reproduzir o mesmo padrão da versão serial.